BÀI GIẢNG HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

CHUYÊN ĐỀ: CON TRỎ – CẤP PHÁT ĐỘNG – ỨNG DỤNG

TỔNG QUAN NỘI DUNG BÀI HỌC

1. Biến và vùng nhớ RAM
2. Toán tử & và *
3. Khái niệm con trỏ (Pointer)
4. Cú pháp khai báo con trỏ
5. Khởi tạo con trỏ và con trỏ NULL
6. Kích thước con trỏ và cấp phát bộ nhớ
7. Con trỏ và hàm
8. Con trỏ và chuỗi ký tự
9. Con trỏ trong mảng một chiều
10. Con trỏ và kiểu cấu trúc (Struct)
11. Hệ thống bài tập ứng dụng

---

1. Biến và Vùng Nhớ RAM

1.1. Vấn đề dẫn nhập: Quản lý biến trong bộ nhớ

Xét chương trình C++ sau:

```cpp
#include <iostream>
using namespace std;

int main()
{
    char c = 'F';
    int x = 10;
    float y = 1.8;

    return 0;
}
```

Đặt vấn đề: Trong quá trình chương trình vận hành, các biến `c`, `x`, `y` được hệ điều hành và phần cứng quản lý, lưu trữ trong bộ nhớ như thế nào?

1.2. Đặc tính của vùng nhớ RAM (Random Access Memory)

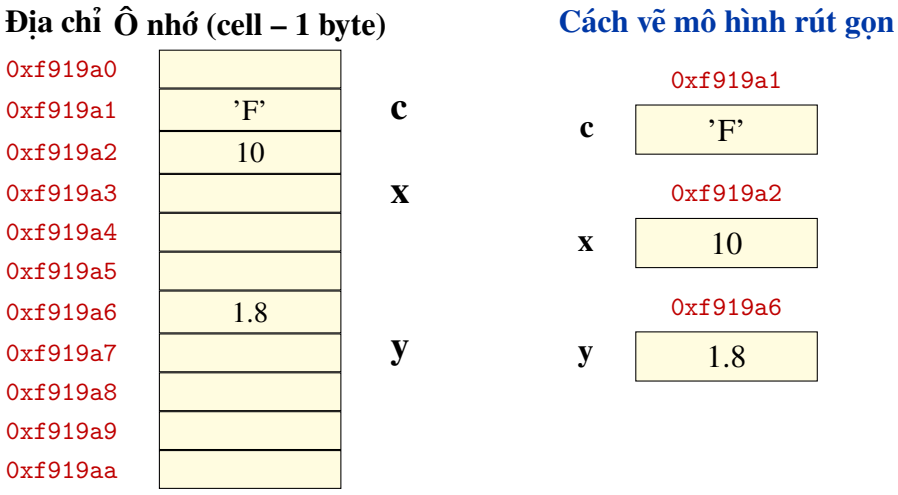
RAM là bộ nhớ truy xuất ngẫu nhiên chính của máy tính, có các đặc điểm cốt lõi sau:

- **Mục đích sử dụng:** Được dùng để lưu trữ dữ liệu và chương trình đang hoạt động trong thời gian thực.
- **Cấu trúc ô nhớ (cell):** Trong RAM có rất nhiều ô nhớ (*memory cell*), mỗi cell thông thường có kích thước **1 byte** (8 bit).
- **Địa chỉ ô nhớ:** Mỗi cell trong RAM sở hữu một **địa chỉ duy nhất** để định vị và truy cập (thường biểu diễn ở hệ cơ số 16 – Hexadecimal, ví dụ: `0xf919a0`, `0xf919a1`, ...).
- **Truy xuất ngẫu nhiên và nhanh chóng:** Cho phép CPU truy cập bất kỳ vị trí ô nhớ nào trong RAM một cách ngẫu nhiên và rất nhanh chóng, với thời gian truy xuất đồng đều.
- **Tính chất tạm thời (Volatile):** RAM là vùng nhớ khả biến; dữ liệu chỉ tồn tại trong khi máy tính đang hoạt động và bị xóa sạch khi máy tính tắt nguồn.

1.3. Cơ chế phân bổ ô nhớ của biến trên RAM

Mỗi biến khi khai báo sẽ chiếm giữ một số lượng cell nhớ tương ứng với kích thước kiểu dữ liệu của nó:

- Biến `char c = 'F'`: Kiểu `char` chiếm **1 byte** (1 cell tại địa chỉ `0xf919a1`).
- Biến `int x = 10`: Kiểu `int` chiếm **4 byte** (4 cells: `0xf919a2`, `0xf919a3`, `0xf919a4`, `0xf919a5`). Địa chỉ đại diện cho biến `x` là địa chỉ bắt đầu: `0xf919a2`.
- Biến `float y = 1.8`: Kiểu `float` chiếm **4 byte** (4 cells: `0xf919a6`, `0xf919a7`, `0xf919a8`, `0xf919a9`). Địa chỉ đại diện cho biến `y` là địa chỉ bắt đầu: `0xf919a6`.



Hình 1: Cơ chế tổ chức dữ liệu các biến trên vùng nhớ RAM.

2. Các Toán Tử & và *

2.1. Định nghĩa hai toán tử cơ bản

- **Toán tử lấy địa chỉ** (`&`):

- *Chức năng:* Dùng để lấy địa chỉ ô nhớ trong bộ nhớ của biến.
- *Phương pháp dùng:* `&tên_biến`
- *Ví dụ:* Nếu biến `x` có giá trị `36` và nằm tại địa chỉ `0xb369`, thì `&x` trả về `0xb369`.

- **Toán tử giải tham chiếu / truy xuất giá trị (`*`):**
  - *Chức năng:* Dùng để truy cập vào giá trị mà một con trỏ đang trỏ đến (hoặc giá trị mà một địa chỉ ô nhớ đang quản lý).
  - *Phương pháp dùng:* `*địa_chỉ_biến` hoặc `*tên_con_trỏ`
  - *Ví dụ:* Biểu thức `*(&x)` sẽ truy cập vào ô nhớ tại địa chỉ `&x` để lấy lại giá trị gốc `36`.

2.2. Chương trình minh họa toán tử `&` và `*`

```cpp
#include <iostream>
#include <cstdio>
using namespace std;

int main()
{
    int x = 36; // Gia su x duoc cap phat tai o nho co dia chi 0xb369

    // In gia tri va dia chi bang cout
    cout << "Gia tri bien x = " << x << endl;        // In: 36
    cout << "Dia chi bien x : " << &x << endl;       // In: dia chi hexa (vi du:
    0xb369)

    // In dia chi bang printf
    printf("Dia chi bien x : %p\n", (void*)&x);      // Dinh dang con tro chuan
    (%p)
    printf("Dia chi bien x : %x\n", &x);             // Dinh dang so nguyen he
    16 (%x)

    // Truy xuat gia tri thong qua toan tu giai tham chieu *(&x)
    cout << "Gia tri bien x = " << *(&x) << endl;    // In: 36

    return 0;
}
```

3. Khái Niệm Con Trỏ (Pointer)

- **Định nghĩa:** Con trỏ (*pointer*) là một **kiểu dữ liệu đặc biệt** dùng để **lưu địa chỉ** của một ô nhớ khác trong bộ nhớ.
- **Tính năng:** Con trỏ cho phép chương trình truy cập và thay đổi dữ liệu của biến được quản lý bởi một địa chỉ cụ thể.
- **Lưu ý sử dụng:** Thao tác với con trỏ can thiệp trực tiếp vào bộ nhớ, do đó cần cẩn thận và chắc chắn sử dụng chính xác để tránh các vấn đề liên quan đến quản lý bộ nhớ (*memory leak, dangling pointer, crash*).

4. Cú Pháp Khai Báo Con Trỏ

4.1. Cú pháp chung

```
Kiểu_Dữ_Liệu* tên_con_trỏ;
```

Trong đó:

- `Kiểu_Dữ_Liệu`: Là kiểu dữ liệu của biến mà con trỏ sẽ trỏ tới (ví dụ: `int`, `float`, `char`, `double`, `SinhVien`, ...).
- `*`: Toán tử con trỏ, đặt sau kiểu dữ liệu để khai báo là một con trỏ.
- `tên_con_trỏ`: Tên của biến con trỏ.

4.2. Các ví dụ khai báo

```
int* iP;              // Con tro iP tro toi mot bien kieu int
float* fP;            // Con tro fP tro toi mot bien kieu float

// Khai bao nhieu con tro tren mot dong: phai co dau * truoc tung ten bien
int *p1, *p2, *p3;

// Con tro tro toi kieu du lieu cau truc (struct)
SinhVien* pSV;        // Con tro tro toi cau truc SinhVien
ToaDo* pTD;           // Con tro tro toi cau truc ToaDo
```

5. Khởi Tạo Con Trỏ và Con Trỏ NULL

5.1. Các trạng thái khởi tạo của con trỏ

1. **Con trỏ chưa khởi tạo:** Khi khai báo kiểu con trỏ và không gán giá trị địa chỉ nào hết thì con trỏ đó **chưa khởi tạo** (chứa giá trị rác ngẫu nhiên, ví dụ: `0xb577`). Truy xuất vùng nhớ này rất nguy hiểm:

```
int* p1; // Con tro chua khoi tao (chua dia chi rac)

```

2. **Con trỏ NULL:** Nếu gán giá trị `NULL` cho con trỏ thì gọi là **con trỏ NULL**, nó sẽ không trỏ vào địa chỉ của biến nào cả:

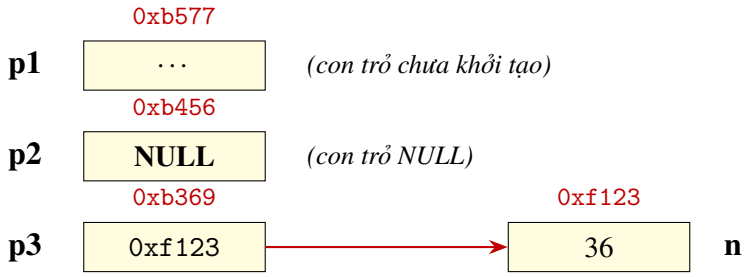
```
int* p2 = NULL; // Con tro NULL (an toan khi kiem tra dieu kien)

```

3. **Con trỏ trỏ đến địa chỉ của biến đã có:**

```
int n = 36;        // Bien n tai dia chi 0xf123, gia tri 36
int* p3 = &n;      // Con tro p3 tai dia chi 0xb369, chua gia tri la 0xf123

```



Hình 2: Biểu diễn trạng thái của các con trỏ trong bộ nhớ.

6. Kích Thước Con Trỏ và Cấp Phát Bộ Nhớ

6.1. Xác định kích thước bằng toán tử `sizeof`

Để xác định kích thước của **kiểu dữ liệu** hoặc **biến** thì dùng `size` hoặc `sizeof()`. Đơn vị: **byte**.

- *Cú pháp:* `sizeof(kiểu_dữ_liệu)` hoặc `sizeof(tên_biến)` (hoặc `sizeof tên_biến`).

```
double x = 9.6;
double* p;

cout << sizeof(int) << endl;      // In: 4 (byte)
cout << sizeof(double) << endl;   // In: 8 (byte)
cout << sizeof x << endl;         // In: 8 (byte)
cout << sizeof p << endl;         // In: 4 (he 32-bit) hoac 8 (he 64-bit)
```

Quy tắc quan trọng về kích thước con trỏ:

- Kích thước của một con trỏ trong C++ có thể khác nhau trên các nền tảng và trình biên dịch khác nhau:
  - Hệ thống **32-bit**: Con trỏ có kích thước **4 byte**.
  - Hệ thống **64-bit**: Con trỏ có kích thước **8 byte**.
- Kích thước này chỉ phụ thuộc vào kiến trúc hệ thống, không phụ thuộc vào kiểu dữ liệu được trỏ tới.

6.2. Phân biệt Cấp phát tĩnh và Cấp phát động

- **Cấp phát bộ nhớ tĩnh (Static Memory Allocation):**
  - Được thực hiện bằng cách khai báo **mảng** hoặc **biến** với kích thước cụ thể **tại thời điểm biên dịch**.
  - Các biến và mảng này tồn tại trong suốt thời gian chạy chương trình và **không thay đổi kích thước** khi chạy.
- **Cấp phát động (Dynamic Memory Allocation):**
  - Cho phép cấp phát bộ nhớ trong quá trình chạy chương trình (*runtime*) và giải phóng bộ nhớ khi không cần sử dụng nữa.

Tiêu chí	Cấp phát bộ nhớ tĩnh	Cấp phát bộ nhớ động
Thời điểm thực hiện	Tại thời điểm biên dịch (*Compile-time*)	Trong khi chương trình chạy (*Runtime*)
Vùng nhớ lưu trữ	Vùng nhớ Stack / Data Segment	Vùng nhớ Heap
Kích thước vùng nhớ	Cố định, không thể thay đổi	Co giãn linh hoạt theo nhu cầu
Cơ chế thu hồi	Tự động thu hồi khi hết phạm vi	Lập trình viên chủ động giải phóng

Bảng 1: So sánh giữa cấp phát bộ nhớ tĩnh và cấp phát bộ nhớ động.

6.3. Phương pháp cấp phát bộ nhớ trong C và C++

Ngôn ngữ	Cấp phát bộ nhớ	Thu hồi – Giải phóng
C++	`new`	`delete` / `delete[]`
C	`malloc()`, `calloc()`, `realloc()`	`free()`

Bảng 2: Các cơ chế cấp phát động tương ứng trong C++ và C.

6.4. Cấp phát động một biến đơn bằng `new`

```cpp
#include <iostream>
using namespace std;

int main()
{
    int* p = new int; // Cap phat dong 1 o nho kieu int tren Heap

    if (p == NULL) // Kiem tra cap phat co thanh cong khong
    {
        cout << "Cap phat khong thanh cong" << endl;
        return 1;
    }

    *p = 6789; // Gan gia tri vao o nho duoc tro toi

    return 0;
}
```

6.5. Vừa cấp phát động vừa khởi tạo giá trị

Cú pháp:

```
Kiểu_Dữ_Liệu* tên_p = new Kiểu_Dữ_Liệu(giá_trị);
```

Ví dụ:

```cpp
int* p = new int(6789); // Vua cap phat vua khoi tao gia tri 6789
cout << *p;             // In ra: 6789
```

6.6. Thu hồi vùng nhớ và Hiện tượng Lạc con trỏ (Dangling Pointer)

- Dùng toán tử `delete` để thu hồi vùng nhớ đã cấp phát: `delete p;`
- Sau khi thu hồi vùng nhớ bằng `delete`, **con trỏ vẫn trỏ đến vùng nhớ đó**.
- Có thể gọi tham chiếu nhưng kết quả không lường trước được ⇒ **Hiện tượng lạc con trỏ** (*dangling pointer*).

```
int* p = new int(6789);
cout << "Gia tri p = " << p << endl;      // Vi du in: 0xe91970
cout << *p << endl << endl;               // In: 6789

delete p; // Thu hoi vung nho da cap phat

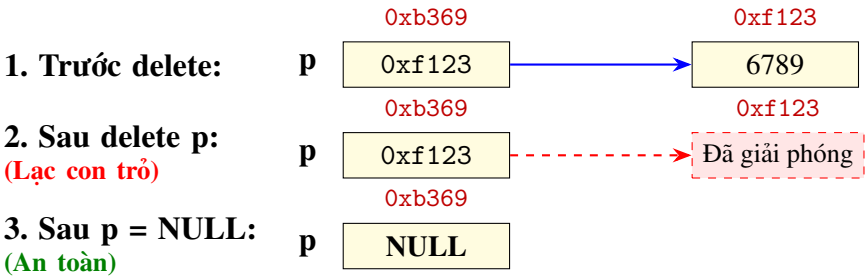
cout << "Gia tri p = " << p << endl;      // p van giu dia chi 0xe91970!
cout << *p << endl;                       // In gia tri rac: 15302288 hoac
    16023184!
```

6.7. Giải pháp xử lý con trỏ bị thất lạc

Để tránh con trỏ bị thất lạc thì sau khi `delete`, ta gán ngay giá trị `NULL` cho con trỏ:

```
int* p = new int(6789);
delete p;
p = NULL; // Con tro p duoc gan ve NULL an toan

cout << "Gia tri p = " << p << endl; // In: 0 (NULL)
// cout << *p << endl; // Khong truy xuat *p khi p la NULL!
```



Hình 3: Cơ chế thu hồi vùng nhớ và xử lý hiện tượng lạc con trỏ.

7. Con Trỏ và Hàm

Con trỏ có thể dùng làm **tham số của hàm** hoặc **kiểu dữ liệu** trả về của hàm.

7.1. Con trỏ làm kiểu dữ liệu trả về của hàm

```
int* input()
{
    int* pi = new int; // Cap phat dong o nho tren Heap
    cout << "Nhap gia tri: ";
    cin >> *pi;
```

```
    return pi;          // Tra ve con tro
}

int main()
{
    int* p = input();
    cout << "Gia tri = " << *p << endl;

    delete p;           // Thu hoi o nho sau khi su dung
    p = NULL;
    return 0;
}
```

7.2. Con trỏ làm tham số của hàm (Hàm hoán vị)

```
void hoan_vi(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x = 3, y = 6;
    cout << "Truoc hoan vi. x = " << x << "    y = " << y << endl;

    hoan_vi(&x, &y); // Truyen dia chi cua cac bien vao ham

    cout << "Sau hoan vi.  x = " << x << "    y = " << y << endl;
    return 0;
}
```

8. Con Trỏ Trong Mảng Một Chiều

8.1. Các địa chỉ trong mảng một chiều

```
int a[] = {5, 3, 7, 5, 9, 1};
cout << "Dia chi mang: " << a << endl;
for (int i = 0; i < 6; i++)
    cout << i << "  " << &a[i] << endl;

double b[] = {5.1, 3.6, 7.9, 5.3, 9.3, 1.9};
cout << "Dia chi mang: " << b << endl;
for (int i = 0; i < 6; i++)
    cout << i << "  " << &b[i] << endl;
```

Quy luật phân bổ địa chỉ:

- Tên mảng đại diện cho địa chỉ phần tử đầu tiên: `a == &a[0]`.
- Mảng `int`: Mỗi phần tử cách nhau **4 byte** (`0x61fea0`, `0x61fea4`, ...).

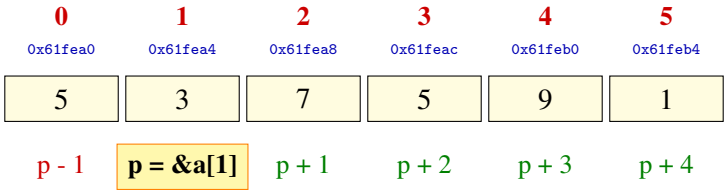
- Mảng `double`: Mỗi phần tử cách nhau **8 byte** (`0x61fe70`, `0x61fe78`, ...).

8.2. Phép toán số học trên địa chỉ con trỏ (Pointer Arithmetic)

- Khi con trỏ **cộng/trừ với một số nguyên**, địa chỉ của con trỏ sẽ **tăng/giảm** lên một số lượng bộ nhớ tương ứng với **số nguyên nhân vào kích thước của kiểu dữ liệu** mà con trỏ đang trỏ tới:

$$\boxed{\text{Địa chỉ mới} = \text{Địa chỉ cũ} \pm (n \times \text{sizeof}(\text{Kiểu_Dữ_Liệu}))}$$

- Cho mảng `int a[] = {5, 3, 7, 5, 9, 1};` và con trỏ `p = &a[1];`:
  - `*(p - 1)` trỏ về `a[0]`, có giá trị là 5.
  - `*(p + 1)` trỏ tới `a[2]`, có giá trị là 7.
  - Tổng quát: Biểu thức `*(a + i)` hoàn toàn tương đương với `a[i]`.



Hình 4: Mô hình mảng một chiều trong bộ nhớ và bước nhảy địa chỉ con trỏ.

8.3. Duyệt mảng bằng con trỏ

```
int a[] = {5, 3, 7, 5, 9, 1};

// Cach 1: In dia chi cac phan tu trong mang
for (int i = 0; i < 6; i++)
    cout << a + i << endl;      // In dia chi

// Cach 2: In gia tri cac phan tu bang toan tu giai tham chieu *
for (int i = 0; i < 6; i++)
    cout << *(a + i) << endl;  // Tuong duong cout << a[i];
```

8.4. Các hàm xử lý mảng một chiều áp dụng kỹ thuật con trỏ

8.4.1. Hàm xuất mảng

```
void xuat(int* a, int n)
{
    for (int i = 0; i < n; i++)
        cout << *(a + i) << endl;
}
```

8.4.2. Hàm khởi tạo mảng tĩnh

```cpp
void khoiTao(int* a, int& n)
{
    n = 6;
    *(a + 0) = 3;
    *(a + 1) = 1;
    *(a + 2) = 6;
    *(a + 3) = 9;
    *(a + 4) = 7;
    *(a + 5) = 8;
}
```

8.4.3. Hàm nhập mảng từ bàn phím

```cpp
void nhap(int* a, int& n)
{
    cout << "So luong can nhap: ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "\tNhap a[" << i << "] = ";
        cin >> *(a + i); // Hoac cin >> a[i];
    }
}
```

8.4.4. Hàm tính tổng các phần tử

- **Input:** Mảng `a` (`int* a`), Số lượng phần tử `n` (`int n`).
- **Output:** Tổng các phần tử (trả về kiểu `int`).

```cpp
int tinhTong(int* a, int n)
{
    int t = 0;
    for (int i = 0; i < n; i++)
        t = t + *(a + i);
    return t;
}
```

8.5. Cú pháp cấp phát bộ nhớ động cho mảng một chiều

Cú pháp:

```
Kiểu_Dữ_Liệu* tên_mảng = new Kiểu_Dữ_Liệu[kích_thước];
```

Ví dụ thực thi và giải phóng:

```cpp
int main()
{
    int* a = new int[100]; // Cap phat mang dong 100 so nguyen tren Heap
    int n = 0;

    khoiTao(a, n);
```

```
    xuat(a, n);

    delete[] a; // Thu hoi mang dong bang delete[]
    a = NULL;
    return 0;
}
```

8.6. Bài tập trắc nghiệm phân tích (Đề thi)

```
#include <iostream>
using namespace std;

int main()
{
    int a[] = {1, 3, 5, 7, 9, 2, 4, 6, 8};
    cout << *(a + 3) + *(a + 7);
    return 0;
}
```

Giải thích đáp án:

- `*(a + 3)` lấy giá trị phần tử tại chỉ số 3: $a[3] = 7$.
- `*(a + 7)` lấy giá trị phần tử tại chỉ số 7: $a[7] = 6$.
- Kết quả: $7 + 6 = 13$. Chương trình xuất ra giá trị 13.

9. Con Trỏ và Kiểu Cấu Trúc (Struct)

9.1. Cú pháp khai báo và cấp phát con trỏ cấu trúc

- **Khai báo con trỏ:** `Kiểu_Cấu_Trúc* tên_biến;`
- **Cấp phát bộ nhớ:** `tên_biến = new Kiểu_Cấu_Trúc();`
- **Vừa khai báo vừa cấp phát:** `Kiểu_Cấu_Trúc* tên_biến = new Kiểu_Cấu_Trúc();`

9.2. Truy xuất thuộc tính bằng toán tử mũi tên (->)

`tên_biến_con_trỏ->tên_thuộc_tính`

Ví dụ:

```
struct Diem
{
    int x;
    int y;
};

int main()
{
    Diem a = {3, 5};
    Diem* p = &a; // Con tro p tro toi cau truc a
```

```
11
12      cout << "Hoanh do x = " << p->x << endl; // In: 3
13      cout << "Tung do y = " << p->y << endl;  // In: 5
14      return 0;
15  }
```

9.3. Quan hệ tương đương giữa -> và (*p).

Hai cách truy xuất thành phần thông qua con trỏ là hoàn toàn tương đương nhau:

$$\texttt{p->x} \iff \texttt{(*p).x}$$

```
1   void xuat(Diem d)
2   {
3       cout << "(" << d.x << "; " << d.y << ")\n";
4   }
5
6   int main()
7   {
8       // Cach 1: Dung toan tu ->
9       Diem* b = new Diem;
10      b->x = 6;
11      b->y = 8;
12      xuat(*b); // In: (6; 8)
13
14      // Cach 2: Dung giai tham chieu ket hop dau cham .
15      Diem* c = new Diem;
16      (*c).x = 7;
17      (*c).y = 9;
18      xuat(*c); // In: (7; 9)
19
20      delete b; b = NULL;
21      delete c; c = NULL;
22      return 0;
23  }
```

9.4. Vừa cấp phát vừa khởi tạo giá trị cho cấu trúc

```
1   Diem* d = new Diem({1, 3});
2   xuat(*d); // In: (1; 3)
3
4   delete d;
5   d = NULL;
```

9.5. Cấp phát mảng động các phần tử cấu trúc

```
1   void khoiTao(Diem* d, int& n)
2   {
3       n = 3;
4       *(d + 0) = {1, 2};
5       *(d + 1) = {2, 6};
```

```
    *(d + 2) = {3, 1};
}

void xuatMang(Diem* d, int n)
{
    for (int i = 0; i < n; i++)
        xuat(*(d + i));
}

int main()
{
    Diem* arr = new Diem[100]; // Mang dong chua 100 diem
    int n = 0;

    khoiTao(arr, n);
    xuatMang(arr, n);

    delete[] arr;
    arr = NULL;
    return 0;
}
```

9.6. Cấu trúc tự tham chiếu (Đệ quy tự trỏ – Self-referential Structures)

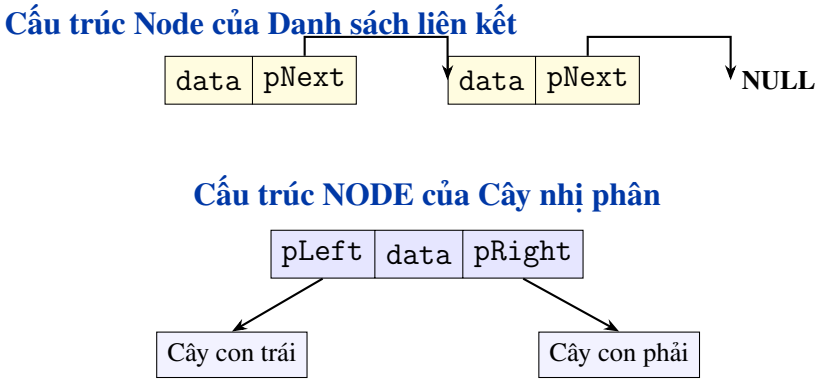
Con trỏ cấu trúc là nền tảng cốt lõi để xây dựng các cấu trúc dữ liệu liên kết động:

- **Cấu trúc Node của Danh sách liên kết (Linked List):**

```
struct Node
{
    int data;     // Du lieu cua nut
    Node* pNext;  // Con tro tro toi Node ke tiep
};
```

- **Cấu trúc NODE của Cây nhị phân (Binary Tree):**

```
struct NODE
{
    int data;     // Du lieu cua nut
    NODE* pLeft;  // Con tro tro toi cay con ben trai
    NODE* pRight; // Con tro tro toi cay con ben phai
};
```



Hình 5: Mô hình minh họa các cấu trúc dữ liệu tự tham chiếu.

10. Hệ Thống Bài Tập Ứng Dụng

10.1. Bài Tập 1: Áp dụng con trỏ cho mảng một chiều

Đề bài: Xây dựng mảng một chiều lưu trữ các số thực. Sau đó, viết các hàm thực hiện các chức năng sau:

1. Tạo giá trị ngẫu nhiên (số thực, lấy 2 chữ số sau dấu phẩy thập phân) thuộc khoảng $(-268; 339)$. Số lượng phần tử thuộc đoạn $[10; 20]$.
2. Xuất (in) mảng vừa tạo ra màn hình.
3. Tính tổng các giá trị có trong mảng.
4. Tính tổng và tích các giá trị có trong mảng (viết gom trong 1 hàm duy nhất).
5. Đếm số lần xuất hiện của một phần tử x bất kỳ trong mảng.
6. Trả về các phần tử chẵn có trong mảng (tính chẵn – lẻ dựa vào giá trị nguyên của số thực).
7. Tìm vị trí (chỉ số) của phần tử nhỏ nhất trong mảng.
8. Tạo một mảng mới chứa các phần tử đảo ngược từ mảng ban đầu.
9. Trả về danh sách các số âm có trong mảng.
10. Trả về số lượng các giá trị trong mảng thuộc đoạn $[x, y]$.
11. Trả về danh sách các giá trị trong mảng thuộc đoạn $[x, y]$.

10.2. Bài Tập 2: Kỹ thuật cấp phát động cho mảng một chiều

Đề bài: Áp dụng kỹ thuật cấp phát bộ nhớ động cho mảng một chiều. Viết các **HÀM** thực hiện các yêu cầu sau:

1. Khởi tạo mảng số thực động gồm các giá trị mẫu:

   `| 0.8 | 5.6 | 9.1 | 7.3 | 10 | 5.9 | 7.2 | 9.3 | 8.0 | 8.7 |`

2. Xuất mảng ra màn hình.
3. Kiểm tra mảng có toàn số dương hay không.

4. Kiểm tra mảng có đối xứng hay không.
5. Kiểm tra trong mảng có chứa số nguyên tố, số chính phương hay không.
6. Tìm kiếm một giá trị có trong mảng.
7. Tìm các số nguyên tố có trong mảng.
8. Tìm các số chính phương có trong mảng.
9. Trả về các số âm trong mảng.
10. Tìm các số dương trong mảng và tính tổng của chúng.
11. Đảo ngược các phần tử trong mảng trực tiếp trên mảng động.

10.3. Bài Tập 3: Quản lý và tính tiền lương nhân viên bằng cấp phát động

Mô tả bài toán: Viết chương trình quản lý và tính tiền lương cho các nhân viên trong một công ty.

- **Thông tin mỗi nhân viên:** Mã nhân viên, Họ tên, Lương cơ bản, Số ngày làm việc, Lương hằng tháng.
- **Công thức tính lương hằng tháng:**

$$\text{Lương hằng tháng} = \text{Lương căn bản} + \text{Số ngày làm việc} \times 180.000 \text{ đ}$$

- **Quy định tiền thưởng và phụ cấp:**
  - Nếu lương hằng tháng $> 8.000.000$ đ thì được **thưởng thêm** 5%.
  - Nếu lương cơ bản $< 5.000.000$ đ thì được **phụ cấp thêm** 10%.

Các yêu cầu xây dựng chương trình (áp dụng cấp phát động):

1. Tạo dữ liệu các nhân viên dùng để kiểm thử (10 nhân viên).
2. Xuất danh sách nhân viên trong công ty.
3. Tính và cập nhật tiền lương cho các loại nhân viên.
4. Tính tổng tiền lương hằng tháng mà công ty phải chi trả cho toàn bộ nhân viên.
5. Tính tổng tiền lương hằng tháng các nhân viên có lương cơ bản $< 5.000.000$ đ.
6. Tìm nhân viên theo mã nhân viên.
7. Tìm các nhân viên có lương hằng tháng thấp nhất.
8. Tìm các nhân viên có lương cơ bản cao nhất.